

Reporte Conceptual

Laboratorio: Procesamiento de Lenguaje Natural con LLMs

Estudiante:	Legin-Pro
Curso:	Inteligencia Artificial
Proveedor LLM:	Groq — llama-3.3-70b-versatile
Repositorio:	github.com/Legin-Pro/lab-pln

1. Introducción

Este laboratorio tiene como objetivo integrar técnicas fundamentales de Procesamiento de Lenguaje Natural (PLN) con Modelos de Lenguaje a Gran Escala (LLMs) mediante la API de Groq. Se implementaron cuatro módulos que cubren las áreas principales del PLN: preprocesamiento de texto, clasificación y reconocimiento de entidades nombradas (NER), análisis de sentimientos temporal y chatbots conversacionales.

El proveedor seleccionado fue Groq por su capa gratuita generosa, baja latencia y soporte para modelos de última generación como LLaMA 3.3 70B, lo que lo hace ideal para proyectos académicos.

2. Investigación de Proveedores LLM

Se investigaron los principales proveedores de LLMs disponibles actualmente. La siguiente tabla compara sus características relevantes:

Proveedor	Modelo destacado	SDK Python	Tier gratuito
Groq	LLaMA 3.3 70B	groq	Sí (generoso)
OpenAI	GPT-4o	openai	No
Anthropic	Claude 3.5 Sonnet	anthropic	No
Google	Gemini 1.5 Flash	google-generativeai	Sí (limitado)
Cohere	Command R+	cohere	Sí
Mistral AI	Mistral Large	mistralai	Sí (limitado)
Together AI	Mixtral 8x7B	together	Créditos iniciales
Hugging Face	Varios (open)	huggingface_hub	Sí

Selección justificada — Groq: Se eligió Groq como proveedor principal debido a su velocidad de inferencia superior (hasta 750 tokens/seg), capa gratuita sin restricciones de tarjeta de crédito, y

soporte para LLaMA 3.3 70B, un modelo de código abierto de alto rendimiento comparable a modelos comerciales.

3. Módulos Implementados

3.1 Módulo 1 — Preprocesamiento Híbrido

Objetivo: Aplicar técnicas clásicas de PLN sobre texto en español y enriquecer los resultados con análisis semántico vía LLM.

Técnicas PLN locales (NLTK):

- Tokenización con `word_tokenize` para separar el texto en unidades léxicas.
- Eliminación de stopwords en español para reducir ruido léxico.
- Stemming con `SnowballStemmer` para reducir palabras a su raíz morfológica.

Integración LLM: Se envió la lista de tokens al modelo LLaMA 3.3 70B mediante un prompt zero-shot solicitando sinónimos y antónimos en formato JSON estructurado.

Prompt Engineering aplicado: Se especificó el formato de salida explícitamente en el prompt para garantizar respuestas parseables, utilizando la técnica de instrucción de salida estructurada.

3.2 Módulo 2 — Clasificación y NER

Objetivo: Clasificar correos electrónicos como spam o legítimos, y extraer entidades nombradas (personas, organizaciones, fechas).

Técnicas PLN locales (regex):

- Expresiones regulares para extracción de fechas en formato español (ej. '12 de junio').
- Regex para identificación de direcciones de correo electrónico.

Integración LLM: Se utilizó few-shot prompting enviando 5 correos al modelo para clasificación binaria (spam/legítimo) y extracción de entidades en JSON. El LLM demostró comprensión contextual superior al regex para NER en texto libre.

3.3 Módulo 3 — Análisis de Sentimientos Temporal

Objetivo: Analizar la evolución del sentimiento en opiniones de clientes a lo largo del tiempo y visualizar la tendencia.

Integración LLM: Se enviaron 7 opiniones con fechas al modelo, solicitando un score numérico entre -1.0 (muy negativo) y 1.0 (muy positivo) para cada una, con salida en JSON.

Visualización: Se generó una gráfica matplotlib con:

- Línea de sentimiento por fecha con marcadores de puntos.
- Zonas de color: rojo para sentimiento negativo, verde para positivo.
- Línea de tendencia polinomial proyectada (color magenta).
- La gráfica fue guardada como `sentimientos.png`.

3.4 Módulo 4 — Chatbot ARIA

Objetivo: Implementar un chatbot conversacional con personalidad definida, historial de conversación y respuestas contextuales.

Diseño del System Prompt: Se definió la personalidad de ARIA (Asistente de Respuesta Inteligente Avanzada) mediante un system prompt que establece su rol, tono y comportamiento esperado.

Gestión del historial: Se implementó un historial de conversación multi-turno que incluye todos los mensajes previos en cada llamada a la API, permitiendo respuestas contextuales coherentes.

4. Técnicas de Prompt Engineering

Técnica	Módulo	Descripción
Zero-shot	Módulo 1	Instrucción directa sin ejemplos: "Devuelve JSON con sinónimos y antónimos"
Few-shot	Módulo 2	Se proporcionan ejemplos de clasificación antes de la tarea real
System Prompt	Módulo 4	Define la personalidad y comportamiento del chatbot ARIA
Output estructurado	Módulos 1-3	Se especifica explícitamente el formato JSON de respuesta esperado

5. Conclusiones

La integración de técnicas clásicas de PLN con LLMs de última generación demuestra un enfoque híbrido poderoso. Las librerías locales como NLTK son eficientes para tareas deterministas (tokenización, stemming), mientras que los LLMs aportan comprensión semántica profunda para tareas como NER contextual, análisis de sentimientos matizados y generación de lenguaje natural.

Groq demostró ser una excelente opción para entornos académicos: su capa gratuita permitió completar todos los módulos sin costo, y su velocidad de respuesta facilitó el desarrollo iterativo. El modelo LLaMA 3.3 70B mostró capacidad de seguir instrucciones JSON de forma consistente, lo cual es crítico para aplicaciones que requieren salidas estructuradas.

El uso de Prompt Engineering estructurado (especificando el formato JSON en el prompt) fue determinante para obtener respuestas parseables de forma confiable. Esta técnica es fundamental en aplicaciones de producción donde la salida del LLM debe procesarse programáticamente.